

Next 2 Hours — Guided Session

August 2026

Contents

Next 2 Hours — Guided Session	1
Block 1 — Python core, hands-on (0:00-0:45)	1
Block 2 — Finish the drills (0:45-1:00)	2
Block 3 — OOP and dataclasses (1:00-1:35)	2
Block 4 — Inference engine domain (1:35-2:00)	2
What you'll have after 2 hours	3
Next session (whenever you get it)	3

Next 2 Hours — Guided Session

A minute-by-minute session you can start now. Every block says what to open, what to do, and how to know you're done. Keep a terminal and a REPL open.

Setup (2 min). Open a terminal in the folder with these files and run `python3 --version` (need 3.9+). Then `python3` for a REPL in a second tab.

Block 1 — Python core, hands-on (0:00-0:45)

Open: `2_python_syntax_reference.pdf` (or the `.md`).

Type every snippet — don't read passively. Cover these sections in order:

Min	Section	Done when you can...
0-8	§2 Numbers	explain why <code>-7 // 2 == -4</code> and write ceiling division two ways
8-15	§3 Strings	format a float to 2dp and pad a string, from memory
15-25	§4-6 Containers	choose list vs. deque vs. dict vs. set by cost, not habit
25-33	§7-8 Control flow, comprehensions	write a nested comprehension and a generator expression
33-45	§10 Sorting	sort by one key desc + two keys asc, and pick a min with a tie-break

§10 is the highest-value part of this entire session. Do not shortchange it.

Checkpoint (last 5 min of the block): run `python day1_drills.py` and complete **c1-c4**. If any takes over 3 minutes, reread that section now.

Reference links (use only if something doesn't click — don't browse):

- [Python Tutorial §5 Data Structures](#) — official, lists/dicts/sets/comprehensions
 - [Sorting HOW TO](#) — official; the `key=` section is exactly §10
 - [collections module](#) — deque, defaultdict, Counter
 - [heapq module](#) — priority queues
-

Block 2 — Finish the drills (0:45-1:00)

Complete **c5-c8** in `day1_drills.py`: in-place mutation, grouping with defaultdict, Counter, and heap pops with tie-breaks. Check against `day1_drills_answers.py`.

These four mirror operations you will almost certainly write in the OA: draining finished items from a running list, grouping requests by worker, and popping the next event from a heap deterministically.

Done when: `python day1_drills.py` reports 8/10 (c9-c10 are OOP, next).

Block 3 — OOP and dataclasses (1:00-1:35)

Open: `3_oop_refresher.pdf`.

- **1:00-1:15 — writing classes.** `__init__` and `self`, instance vs. class attributes (the shared-mutable trap), `@property` vs. `method`, `super()`.
- **1:15-1:25 — dataclasses.** Type them out: required vs. default fields, `field(default_factory=list)`, `@property` on a dataclass, a `fresh()` copy method. Then complete drills **c9 and c10**.
- **1:25-1:35 — the reading drill.** Open `mock_oa/engine.py` and take 8 *timed* minutes to write answers to: what does each class own? which members are properties vs. methods? what state am I responsible for updating? where's the mutation risk? Check against the answers at the end of the Day 2 refresher.

Done when: drills report 10/10 and your `engine.py` notes match.

Reference links:

- [Python Tutorial §9 Classes](#) — official
 - [dataclasses module](#) — official reference
 - [Real Python: Data Classes guide](#) — friendlier walkthrough if the official docs feel terse
 - [Real Python: @property](#) — if the property/method distinction is still fuzzy
-

Block 4 — Inference engine domain (1:35-2:00)

Open: `4_inference_engine_primer.pdf`, sections 2-5 and the vocabulary list in section 8. Goal is fluency with the words, not depth: *prefill*, *decode*, *KV cache*, *continuous batching*, *chunked prefill*, *preemption*, *TTFT*, *load balancing*.

Active exercise (last 10 min): on paper, draw the timeline for two requests sharing one worker under continuous batching, with `prefill_rate=4`, memory limit 12:

- R1: arrives `t=0`, prompt 8 tokens, `max_new` 3
- R2: arrives `t=0`, prompt 4 tokens, `max_new` 2

Mark each tick as free → admit → work, and write down when each request gets its first token and finishes. Then check against `mock_oa/tests.py` test `t07` (memory limit 12 — same setup with a third request). If your hand-simulation disagrees with the expected values, find out why *now* — that gap is exactly what costs people the real test.

Reference links (optional, only if curious — none are required):

- [vLLM: PagedAttention announcement](#) — the original explainer
 - [Continuous batching, 23x throughput](#) — Anyscale, the standard reference on batching
 - [Insu Jang: Continuous batching & PagedAttention](#) — clear diagrams
 - [vLLM docs](#) — if you want the production system’s vocabulary
-

What you’ll have after 2 hours

- Python container/sorting fluency restored, verified by 10/10 drills
- Class and dataclass syntax you can produce without pausing
- The one exam skill practiced: reading an unfamiliar class file fast
- Enough domain vocabulary that the OA README won’t slow you down

Next session (whenever you get it)

The timed mock: `mock_oa/`, 45–90 minutes depending on what you have. Read `README.md` → `engine.py` → `tests.py` for the first 8–10 minutes without writing code. Then Level 1, run the grader, then Levels 2–3. Skip Level 4 unless you’re ahead of schedule.

Do not open `SPOILERS.md` before that attempt.